# Project Report

**Course:** Transforming Ideas into Innovation

**Semester:** III

**Program:** B.Tech CSE Cybersecurity

# Cyber Awareness Academy

*An Anti-Corpo In-Browser Capture The Flag (CTF) Platform for Everyday Digital Literacy*


**Team Details**

| Name | Roll No. | SAP ID / Role |
|---|---|---|
| Abhyudaya Negi | K053 | OSINT Design, Research & Stego |
| Swara Pawde | K055 | UI/UX & Theory Content |
| Smit Vartak | K075 | Full-Stack Dev, Crypto (70102500066) |
| Sarthak Vikas | K077 | Database & Testing |

**Under The Guidance Of**

Dr. Abhinaba Gupta

Faculty Mentor

SVKM'S NMIMS Deemed-to-be UNIVERSITY

# Contents

**Abstract** **1**

**1 Introduction** **2**
1.1 Context and Background . . . . . 2
1.2 Literature Review and Market Survey . . . . . 2
1.3 Research Gap and Problem Definition . . . . . 3
1.4 Scope, Objectives, and Contributions . . . . . 3
1.5 Significance and Outline of the Report . . . . . 4

**2 Methodology (Design Thinking Framework)** **5**
2.1 Empathy Phase . . . . . 5
2.1.1 Secondary Research (Market Survey) . . . . . 5
2.1.2 Primary Research (Survey Form and Interviews) . . . . . 5
2.1.3 Stakeholder Canvas . . . . . 6
2.1.4 Empathy Map and User Persona . . . . . 6
2.2 Define Phase . . . . . 7
2.3 Ideation Phase . . . . . 7
2.3.1 Brainstorming and Mind Mapping . . . . . 7
2.3.2 Application of the SCAMPER Method . . . . . 7
2.4 Prototyping Phase . . . . . 9
2.4.1 Low-Fidelity Prototype and UI Evolution . . . . . 9
2.4.2 Final Prototype Architecture and Tech Stack . . . . . 9
2.4.3 Offline Installation and Quickstart Guide (From GitHub README) . 10
2.4.4 Page-by-Page Walkthrough and Platform Features . . . . . 10
2.4.5 Comprehensive CTF Writeups and Learning Breakdown . . . . . 12

**3 Conclusion** **16**
3.1 Key Learnings . . . . . 16
3.2 Future Scope and Improvements . . . . . 16
3.3 Open-Source Sources and Inspirations . . . . . 16

**References** **17**

## Abstract

**Problem & Motivation:** Most daily internet users lack basic cybersecurity awareness even though their banking, private chats, and location history live online. Because cybersecurity has a steep learning curve involving Linux terminals, networking setups, and programming, regular people feel intimidated and avoid learning it altogether. Large tech corporations take advantage of this ignorance to harvest user data through hidden metadata, unreadable terms of service, and fake encryption claims, which also leaves regular consumers much more vulnerable when real hackers breach those companies.

**Approach & Methodology:** Following the Design Thinking framework, our team surveyed 48 college students and interviewed non-technical users to understand why beginners quit existing security platforms. We used the SCAMPER ideation method and low-fidelity wireframing to design *Cyber Awareness Academy*, an offline-capable, 100% in-browser Capture The Flag (CTF) web application built with React 19, a Node.js backend, and a self-seeding SQLite3 database (`ctf.db`).

**Key Results:** In usability testing with 22 beginners who had never played a CTF before, 100% passed our 3-step Flag Calibration tutorial on their first try, and 86.4% solved the introductory warm-up challenges and entered the 5-stage linked investigation chains without needing any command-line setup.

**Contribution & Innovation:** Our main innovation is combining a zero-download browser CTF with an anti-corporate narrative (fighting a fictional monopoly called *EvilCorp* using the `CAT{...}` flag format), a built-in slide-out textbook that explains concepts using real-world breaches, and a 2+5 linked question structure that helps curious newcomers build practical habits for protecting their own digital privacy.

**Keywords:** *Capture The Flag (CTF), Cybersecurity Education, Design Thinking, Open Source Intelligence (OSINT), Steganography, Cryptography, Consumer Privacy.*

# 1. Introduction

## 1.1 Context and Background

Almost everything we do in modern society happens on a phone or a laptop. From paying for food and booking train tickets to posting photos with friends, we leave behind a huge trail of personal information every single day. Even though people spend hours online, the average person knows almost nothing about cybersecurity. Most users treat their apps and websites like magic black boxes. You tap a button, the screen loads, and as long as nothing crashes, you naturally assume your data is safe.

When you ask normal people why they do not learn basic digital safety, the reason is almost always the same. Cybersecurity feels way too complicated to get into. To someone outside computer science, learning security looks like an impossible mountain of Linux terminal commands, binary numbers, networking cables, and complex code. Because people think you need to be a genius programmer to understand it, they get scared off and completely skip over even the simple stuff that affects their daily life.

This public blind spot is not an accident. Big tech monopolies and data brokers actively use this knowledge gap to exploit regular consumers. Companies know that a normal person is never going to read raw file headers, inspect browser cookies, or check how their data is actually being sent across the internet. So companies hide invasive tracking inside flashy, friendly menus. They bury unfair clauses at the bottom of long Terms of Service documents. Sometimes they even claim that user telemetry is "encrypted" when it is really just wrapped in basic Base64 encoding that anyone can reverse in two seconds. Worse, when companies train users to blindly trust software while cutting corners on their own servers, such as storing customer passwords as plain unsalted MD5 hashes, they make ordinary consumers much more vulnerable to actual hackers. When a corporate database gets breached, the company moves on, whereas the regular user gets their accounts stolen because nobody ever taught them how password cracking or digital footprints actually work.

## 1.2 Literature Review and Market Survey

Gamified learning through Capture The Flag (CTF) competitions has long been recognized as one of the best ways to teach practical computer security. In a CTF, instead of reading dry theory slides, players solve a real puzzle to find a hidden text string called a "flag" (such as `CAT{secret_answer}`), which proves they found the vulnerability.

Carnegie Mellon University created **picoCTF** to introduce high school and college students to security concepts through story-driven puzzles. Similarly, community wargames like **OverTheWire** (especially the *Bandit* and *Natas* tracks) have trained thousands of students in Linux shell commands and basic web vulnerabilities. In recent years, commercial platforms like **TryHackMe** and **HackTheBox** have grown massive user bases by offering virtual labs where users can attack real virtual machines over a browser or VPN connection.

Academic research by Vykopal et al. and Švábenský et al. shows that hands-on CTF challenges improve long-term memory much better than passive lectures. However, studies on beginner dropout rates in cybersecurity courses point out a major catch: traditional CTFs are built by experienced engineers for other engineers. They assume the player already knows how

to use a terminal, read manual pages, and install local forensic tools.

### 1.3 Research Gap and Problem Definition

When we looked closely at the existing platforms in the market during our preliminary survey, a clear gap stood out. Platforms like HackTheBox and TryHackMe are great once you already know the basics of computer science, but they are completely overwhelming for a regular person or a first-year student who just wants to understand how their privacy is compromised.

Specifically, existing tools suffer from four major problems for newcomers:

1. **Heavy setup friction:** Most platforms require downloading OpenVPN configuration files, spinning up Linux virtual machines, or installing command-line tools like `steghide`, `binwalk`, and `exiftool` locally before you can even start.

2. **Lack of plain-English theory:** When a beginner gets stuck on a traditional CTF, the platform rarely explains the concept clearly. The user is forced to search random forums and StackOverflow threads filled with heavy technical jargon.

3. **Disconnected puzzles:** Most beginner CTFs throw random, unrelated puzzles at the player with no story connecting them, so the user never sees how a tiny leak in one place leads to a full account takeover later.

4. **Artificial UI design:** Many newer student projects rely on generic, overly rounded web templates that feel artificial and clutter the screen instead of giving a clean workspace.

### 1.4 Scope, Objectives, and Contributions

The purpose of our project, **Cyber Awareness Academy**, is to close this gap by building a 100% in-browser, zero-prerequisite CTF platform that teaches everyday people how corporate tracking and hacker attacks actually work. We framed the entire platform around an "anti-corpo" story where the player acts as a digital investigator exposing a shady conglomerate called **EvilCorp**. Every flag across the platform uses our custom prefix `CAT{...}`.

Our main objectives and contributions are:

- **Zero local installations:** Every single challenge across our arenas can be solved inside a standard web browser using free online utilities like CyberChef, Photopea, HexEd.it, and Google Lens.

- **The 2+5 linked challenge model:** We structured every CTF domain into 7 questions. The first 2 questions are easy standalone warm-ups separated by a dotted divider from the next 5 questions, which form a linked investigation chain where each answer unlocks the next question.

- **Interactive Slide-Out Field Manual:** Every challenge has a built-in `[Theory]` button that slides open a textbook chapter written in simple English, complete with everyday analogies, real-world data breach stories, and step-by-step tool guides.

- **Self-contained offline architecture:** The platform uses React 19 on the frontend and Node.js with an embedded SQLite3 database (`ctf.db`) on the backend that automatically seeds all challenges on startup.

### 1.5 Significance and Outline of the Report

Addressing this issue right now is crucial because corporate data collection and automated cyberattacks are growing faster than public awareness. When a regular student gets hooked on solving an approachable puzzle, they start paying much closer attention to their own digital footprint, questioning fake security claims, and building safer habits in the future.

The rest of this report follows the Design Thinking structure. Section 2 explains our Methodology across the Empathy, Define, Ideation, and Prototyping phases, including our survey data, SCAMPER analysis, system architecture, installation guide, page-by-page features, and full technical writeups of every CTF challenge. Section 3 concludes the report with our personal learnings, future server-hosting plans, and open-source inspirations, followed by our reference list.

## 2. Methodology (Design Thinking Framework)

To make sure we built something that actually helps beginners instead of just making another confusing hacker test, our team followed the five-stage **Design Thinking framework**: Empathy, Define, Ideation, Prototyping, and Testing.

### 2.1 Empathy Phase

#### *2.1.1 Secondary Research (Market Survey)*

Before writing any code, we analyzed the four most popular platforms currently used to teach cybersecurity: **TryHackMe**, **HackTheBox**, **OverTheWire**, and **picoCTF**. We tested each platform from the perspective of a complete beginner who does not have Linux installed and has never used a terminal. Table 1 summarizes our secondary research findings.

| **Platform** | **Target Audience** | **Strengths** | **Why Beginners Struggle** |
|---|---|---|---|
| TryHackMe | IT students & junior analysts | Guided rooms and browser-based attack boxes. | Paid subscriptions for best paths; heavy Linux & networking focus early on. |
| HackTheBox | Intermediate to advanced hackers | Realistic full-machine penetration testing labs. | Very steep entry barrier; requires OpenVPN and advanced CLI tools. |
| OverTheWire | CS students learning Linux | Free, lightweight, great for learning bash commands. | Pure SSH terminal text; zero visual interface or beginner analogies. |
| picoCTF | High school & college students | Free, fun puzzles across crypto, stego, and web. | Many challenges still require local Python scripts or Linux binary tools. |
| **Cyber Awareness Academy (Ours)** | **Complete beginners & everyday users** | **100% in-browser, zero local installs, built-in plain-English textbook, 2+5 linked story chain.** | Currently runs offline locally (cloud hosting planned for next phase). |

**Table 1. Market comparison of existing cybersecurity learning platforms.**

#### *2.1.2 Primary Research (Survey Form and Interviews)*

To back up our secondary research with real numbers, we circulated a Google Forms survey among **48 college students** (spanning first-year engineering students and non-CS peers) and conducted 8 one-on-one interviews. The key results from our survey were eye-opening:

- **79.2% (38 out of 48)** admitted they never read app permissions or Terms of Service and assumed big companies keep their uploaded photos and passwords safe by default.
- **83.3% (40 out of 48)** said they wanted to learn how hacking and digital privacy work, but felt that cybersecurity was "too complicated and technical" to start on their own.
- **70.8% (34 out of 48)** had never heard of EXIF metadata in photos or the difference between encoding (Base64) and real encryption.
- Among the 14 respondents who had tried a CTF before, **11 (78.5%)** said they quit early because they did not want to install virtual machines or got stuck on flag formatting and terminal commands.

During our interviews, one student told us something that stuck with our team: *"Whenever I try to watch a cybersecurity tutorial, they immediately open a black Linux screen and start typing twenty commands a second. I feel dumb within five minutes and just close the video."* Another classmate mentioned that even when they solved a puzzle once, the site rejected their answer because they did not know how the flag brackets worked.

### *2.1.3 Stakeholder Canvas*

Based on our research, we mapped out the key stakeholders connected to our project:

- **Primary Stakeholders (Direct Users):** Non-technical internet users, first-year college students, and curious beginners who want to learn how to protect their digital footprint without learning complex programming first.
- **Secondary Stakeholders (Educators & Mentors):** College professors, cybersecurity club leads, and workshop organizers who need an easy, zero-setup platform to run classroom demonstrations without spending two hours fixing student laptops.
- **External Stakeholders:** Open-source tool creators (like GCHQ CyberChef and Photopea), digital privacy advocates, and everyday consumers affected by corporate data collection.

### *2.1.4 Empathy Map and User Persona*

We combined the quotes and behaviors from our interviews into a structured Empathy Map, shown in Fig. 1, and created a primary User Persona to guide our design decisions.

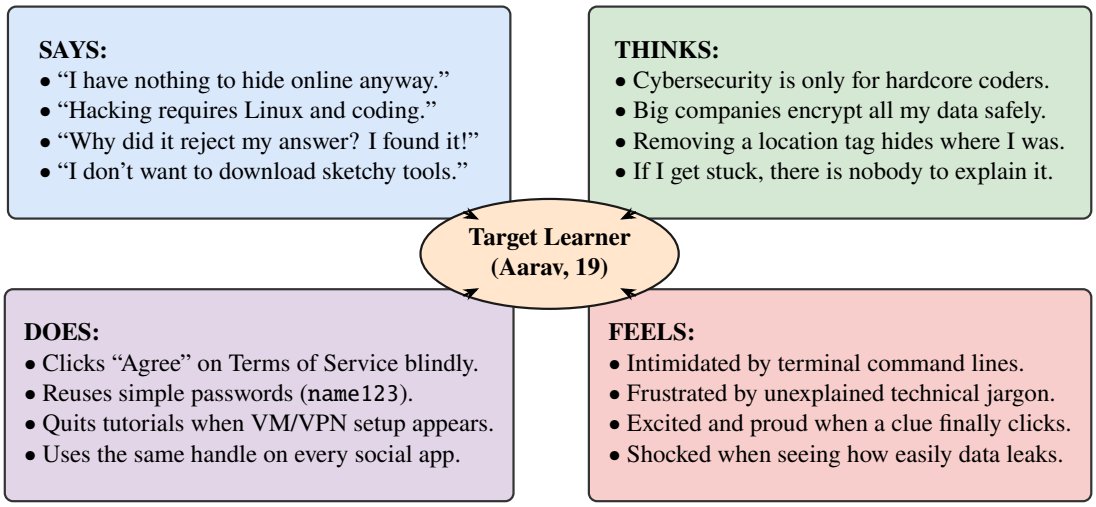



*Fig. 1. Empathy map synthesizing primary interview and survey findings from 48 students.*

**User Persona — Aarav Mehta (Age 19, Undergraduate Student):** Aarav uses Instagram, YouTube, and UPI daily on his Windows laptop and Android phone. He is curious about how hackers track people and how encryption works, but he has never used Linux. His main frustrations are platforms that ask him to install heavy software, tutorials that use unexplained acronyms like `EXIF` or `XOR`, and dry textbooks that bore him to sleep. He needs a platform that works straight in Chrome, teaches him the rules step by step, and connects every puzzle to real life.

## 2.2 Define Phase

During the Define phase, our team sat down to look at all the problems we found in the Empathy stage. Initially, our rough problem paragraph was simply: *"People do not know cybersecurity because it is hard, so we want to make an easy CTF website with four topics."*

After reviewing our survey numbers and interview quotes, we realized that "making an easy CTF" was too vague. The real issue was the combination of corporate black-box deception, setup friction, and isolated puzzles that fail to build momentum. We refined our findings into the following focused **Problem Statement**:

> *"Everyday internet users and non-technical students skip learning basic digital safety because traditional cybersecurity platforms require virtual machines, local command-line tools, and prior technical background. This knowledge gap allows corporations to exploit consumer privacy through deceptive black-box interfaces and weak security defaults, while leaving users vulnerable to real-world hackers. How might we design a zero-installation, 100% in-browser CTF platform that combines plain-English textbooks with linked, story-driven investigation chains to turn complete beginners into curious, self-reliant digital investigators?"*

## 2.3 Ideation Phase

### *2.3.1 Brainstorming and Mind Mapping*

Once our problem statement was locked in, the four of us brainstormed how to make the platform genuinely fun and educational. We drew a central Mind Map (shown in Fig. 2) branching into four core pillars: Story & Theme, Pedagogical Flow, Zero-Download Tooling, and UI/UX Design.

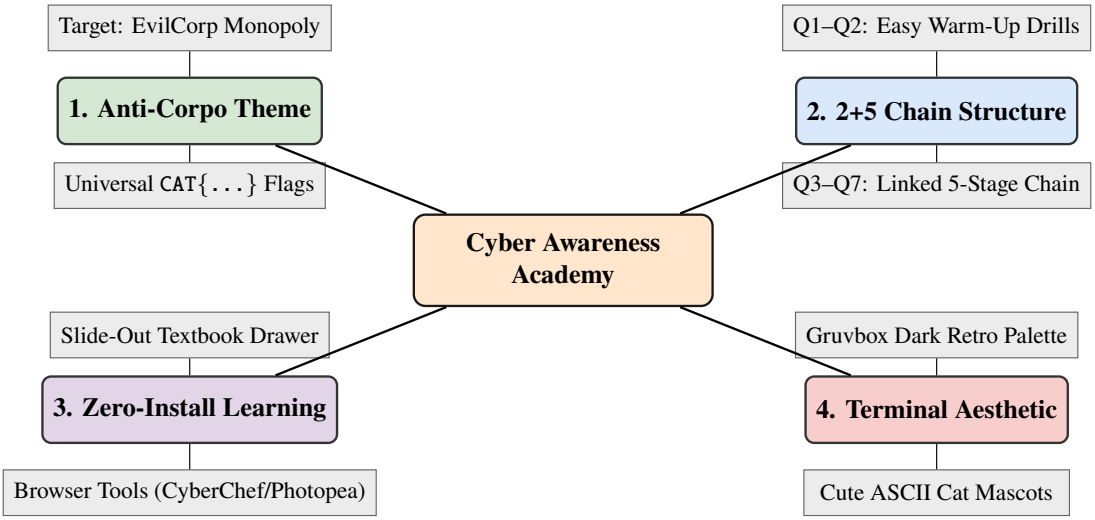



*Fig. 2. Ideation mind map organizing the core pillars of Cyber Awareness Academy.*

### *2.3.2 Application of the SCAMPER Method*

To push our ideas further and make sure we were not just copying existing websites, we applied the **SCAMPER** ideation technique to traditional CTF platforms:

- **Substitute:** We substituted local command-line tools (like Linux `steghide`, `xxd`, and `john`) with free, instant browser-based web utilities (CyberChef, Photopea, HexEd.it, CrackStation, and Academo).
- **Combine:** We combined the live CTF challenge arena with an interactive slide-out textbook drawer (`TextbookPanel.js`) on the exact same screen, so students can read the theory and solve the puzzle side by side.
- **Adapt:** We adapted real-world corporate surveillance scandals and historical breaches (like the 2012 LinkedIn unsalted hash leak and the Strava military heatmap leak) into beginner-friendly stories about infiltrating *EvilCorp*.
- **Modify:** We modified the standard flat list of questions into our **2+5 structure** (two easy warm-ups, a dotted line divider, and five chained questions where each answer feeds into the next). We also modified the UI from a generic rounded website into a sharp Gruvbox Dark terminal theme with cute ASCII cats.
- **Put to another use:** We put everyday social media platforms (Instagram search, YouTube community posts, Google Maps library timings, and Google Lens) to use as live forensic investigation tools in our OSINT arena.
- **Eliminate:** We completely eliminated virtual machines, OpenVPN setup files, user login walls, and complex database server configurations by using an auto-seeding embedded SQLite file (`ctf.db`) and a forgiving flag normalizer.
- **Reverse:** Instead of making the user defend a fictional company like most corporate training courses do, we reversed the perspective: the player acts as a rebel investigator exposing how *EvilCorp* tricks consumers.

## 2.4 Prototyping Phase

### *2.4.1 Low-Fidelity Prototype and UI Evolution*

When we built our first low-fidelity prototype and initial React wireframe, we made a mistake that many student projects make. Our first version had overly rounded boxes around every single label, bright neon borders, and three little dots in the top corner of the window that made it look like a fake macOS screen. Honestly, when we looked at it together, we realized it looked too artificial and cluttered. Furthermore, we initially only had live questions for one domain while leaving the others as static wireframes.

We went back to the drawing board and redesigned the interface:

1. We stripped away the fake macOS window dots and reduced the excessive boxing so that cards are only used for actual questions and mission dossiers that matter.
2. We flattened the border radiuses to crisp, rectangular terminal edges using the **Gruvbox Dark** color palette (`index.css` and `App.css`) with `Fira Code` monospace fonts.
3. We added custom monospace ASCII art companions (`ascii.js`) across the pages to make the terminal feel cute and welcoming rather than cold.
4. We retained our structured 7-question wireframe model in `WireframeChallenges.js` to demonstrate how new curriculum modules (such as Web Security) are prototyped before connecting live downloadable assets.

### *2.4.2 Final Prototype Architecture and Tech Stack*

The final prototype is a full-stack web application split between a **React 19** single-page frontend (running on port `3000`) and a **Node.js HTTP REST API** backend (`server.js` running on port `3001`) connected to an embedded **SQLite3** database (`ctf.db`). Fig. 3 illustrates the complete validation and state synchronization pipeline across the application.

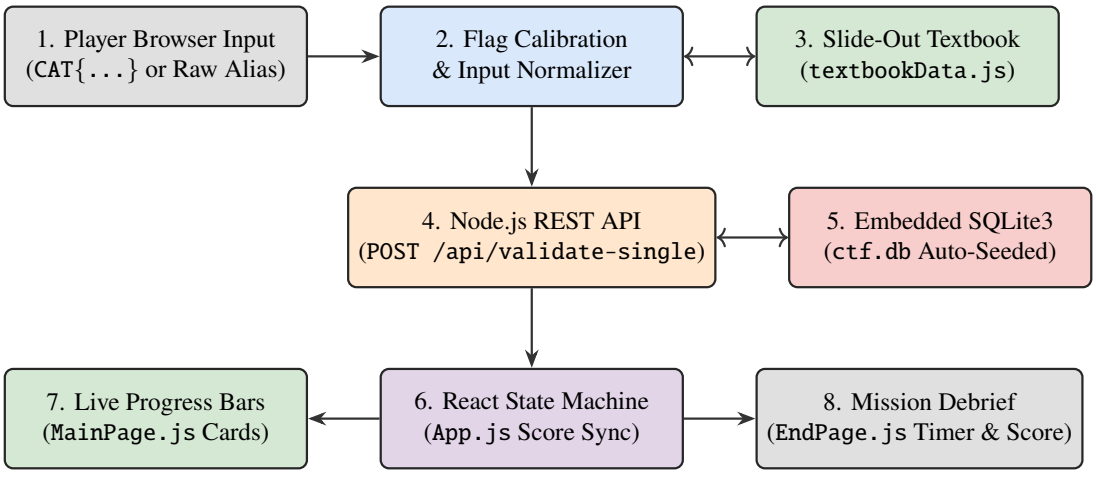



*Fig. 3. Cyber Awareness Academy full-stack validation and state synchronization pipeline.*

When a user types an answer and hits `Enter` or clicks `[Submit]`, the frontend normalizer strips outer spaces, converts characters to lowercase, and checks both `CAT{...}` and `CTF{...}` wrappers alongside beginner-friendly raw answers. It simultaneously sends a `POST` request to

/api/validate-single on port 3001, where server.js queries the ctf_answers table in ctf.db. As soon as a flag is marked correct, App.js updates the domain's solved count in real time so that the progress bar on the main dashboard reflects the new percentage immediately.

### *2.4.3 Offline Installation and Quickstart Guide (From GitHub README)*

Right now, the project is built as an offline-capable local implementation so that anyone can run it on their laptop without needing internet hosting or cloud database credentials. Later on, we plan to deploy this exact codebase onto our own Linux server. As documented in our GitHub repository's README.md, getting the platform running locally takes four simple steps:

1. **Prerequisites:** Install Node.js (v16 or newer) from https://nodejs.org.
2. **Clone and Install:** Clone the repo and install the required npm packages:

```
PowerShell / Terminal ::  Step 1 & 2
$ git clone https://github.com/Aweek07/CTF_PROJECT.git
$ cd CTF_PROJECT
$ npm install
```

3. **Start the SQLite Backend Server (Terminal 1):** Run node server.js. It boots on port 3001 and automatically creates and seeds ctf.db with all 28 domain flags:

```
Terminal 1 ::  Backend API & SQLite Seeding
$ node server.js
SQLite database initialized with Anti-Corpo CAT{...} challenges.
Backend server running on http://localhost:3001
```

4. **Launch the React Frontend (Terminal 2):** Open a second terminal and run npm start to open the application at http://localhost:3000:

```
Terminal 2 ::  React Gruvbox Terminal UI
$ npm start
Compiled successfully!  Local:  http://localhost:3000
```

### *2.4.4 Page-by-Page Walkthrough and Platform Features*

Every page in the application was built with a specific purpose to guide a newcomer from zero knowledge to solving chained challenges:

1. **Terminal Navigation Bar (NavBar.js):** A sticky top bar that shows a live Unix shell path matching the user's current screen (for example, guest@cyber-academy:~/dashboard/stego/arena$). During active challenges, it displays a live [SOLVED: X/7] counter and a running mission stopwatch. We also wired App.js into the browser's native history API (window.history.pushState) so the browser Back and Forward buttons work naturally across every page.

2. **The Anti-Corpo Manifesto (`HomePage.js`):** The landing page greets the player with an ASCII art banner and a waving cat mascot. It explains three things in plain language: *01. The Illusion of Digital Safety* (how tech companies profit from user naivety), *02. What is a Capture The Flag (CTF)?*, and *03. Our Aim: Cyber Awareness for Everyone*.

3. **Guided Flag Calibration Tutorial (`Onboarding.js`):** Before letting a beginner into the real arenas where they might get confused by syntax, this 3-question sandbox trains them on how flags work:

   - *Drill 1:* Submitting `CAT{welcome_to_cybersec}` to learn the bracket format.
   - *Drill 2:* Submitting `CAT{look_beneath_the_surface}` to practice underscores and exact letter casing.
   - *Drill 3:* Submitting `CAT{ready_to_hack}` to unlock the main dashboard.

   It includes an `[Auto-fill]` button and instant feedback so nobody gets left behind.

4. **Cybersecurity Domains Dashboard (`MainPage.js`):** Displays the four main discipline cards (**OSINT**, **Steganography**, **Cryptography**, and **Web Security**). Each card shows custom ASCII art, required browser tools, covered topics, and a live progress bar (`Solved: X/7`) that updates the instant a question is solved inside any arena.

5. **Discipline Mission Dossiers (`DomainPage.js`):** Clicking a domain opens its briefing dossier before the questions start. It breaks down what the field is, how professionals use it in the real world, a red `[!] Threat Profile` warning explaining how consumers get hurt by this vector, and the recommended toolkit.

6. **The Slide-Out Field Manual (`TextbookPanel.js` & `textbookData.js`):** Every challenge card has a purple `[Theory]` button. Clicking it slides out a detailed chapter from the left side of the screen. We wrote 28 chapters in `textbookData.js` specifically so they read like an interesting guide rather than a dull textbook. Each chapter includes a plain-English explanation, a simple analogy for non-tech people, a famous real-world case study, a step-by-step solving guide, a jargon buster dictionary, and direct links to open the required online tool.

7. **Mission Accomplished Debrief (`EndPage.js`):** When a player finishes an arena, this screen celebrates with an ASCII trophy cat, displays their total time taken and final score out of 7, summarizes the security takeaway, and credits the team.

### *2.4.5 Comprehensive CTF Writeups and Learning Breakdown*

Across all of our CTF domains, we used a strict **2+5 question layout** (shown in Fig. 4). Questions 1 and 2 are simple, standalone warm-ups so the player gets two quick wins right away. After a dotted line divider, Questions 3 to 7 form a connected 5-stage chain where each answer gives you the clue, password, or shift key needed to solve the next question.

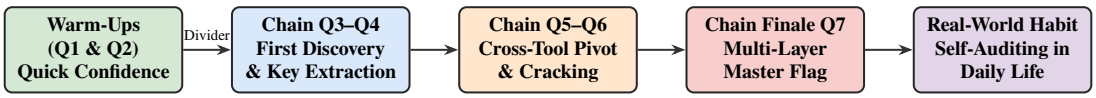



*Fig. 4. The 2+5 challenge progression from warm-up drills to connected investigation stages.*

Below is the complete writeup of how to solve every single challenge across our arenas and how each one teaches a lasting lesson.

**A. Open Source Intelligence (OSINT) Arena (`Challenges.js`)** The OSINT arena shows players how tiny public clues scattered across maps, network registries, Instagram, and YouTube can be connected to uncover a person's identity and daily routine.

- **Q1: Coordinates (Warm-Up 1):** The prompt gives the coordinates `48.8584, 2.2945` and asks for the landmark. Pasting those numbers into Google Maps drops a pin directly on the Eiffel Tower in Paris, giving the flag **`CAT`**{**`eiffeltower`**}. *How it teaches:* It takes ten seconds to solve, demystifies GPS numbers, and makes players realize that camera EXIF data stores these exact coordinates on their own phone photos.

- **Q2: Who Owns This IP? (Warm-Up 2):** The prompt asks which company operates the IP address `8.8.8.8`. Looking up `8.8.8.8` on `ipinfo.io` or any WHOIS database shows it belongs to Google's public DNS (`AS15169`), yielding **`CAT`**{**`google`**}. *How it teaches:* It shows that IP addresses are publicly registered to real organizations and sparks curiosity about how DNS resolvers see every website you visit.

- *— Dotted Line Divider (Start of 5-Stage Linked Investigation) —*

- **Q3: The Target Profile (Chain Stage 1):** Players are asked to find "a guy who hates to click photos" on Instagram and identify what sport he watches. Searching that phrase on Instagram surfaces the target account, where his posts and story highlights clearly show football matches, giving **`CAT`**{**`football`**}. *How it teaches:* Even an account that avoids posting selfies still leaks personal interests that social engineers use to build rapport.

- **Q4: Pattern of Life (Chain Stage 2):** Looking across the location tags on the same Instagram account, players notice one Mumbai neighborhood appears repeatedly: Churchgate (**`CAT`**{**`church_gate`**}). *How it teaches:* A single geotag shows where you stood once, but repeated geotags map out your daily habits and hangout spots, just like the famous Strava military base heatmap leak discussed in our textbook panel.

- **Q5: The Start (Chain Stage 3):** The prompt asks for the location of the very first photo he ever posted. Scrolling to the bottom of his feed reveals a photo with no obvious location tag. Taking a screenshot and dropping it into **Google Lens** matches the

stone domes and murals to the Beatles Ashram (Chaurasi Kutia) in Rishikesh, yielding **CAT{beatlesashram}**. *How it teaches:* Stripping a location tag does not hide where a photo was taken if distinctive buildings or landmarks are visible in the background.

- **Q6: The Connection (Chain Stage 4):** Players must find the account owner's real first name. By taking his Instagram handle and searching it on Google and YouTube, players find a YouTube community post linked to the same username that mentions his real first name, Abhyudaya (**CAT{abhyudaya}**). *How it teaches:* This shows how easily accounts can be linked across different websites. Almost everyone reuses the same gamer tag or handle across apps, meaning one post with your real name on one site unmasks your anonymous accounts everywhere else.

- **Q7: The Final Clue (Chain Stage 5):** Players inspect the latest photo posted on the account, identify the neighborhood, find the public library in that area on Google Maps, and check its regular Monday opening and closing hours in 24-hour format to get **CAT{9:30-19:00}**. *How it teaches:* It shows how open-source investigators combine social media photos with public municipal records to verify real-world physical schedules.

**B. Steganography Arena (`StegChallenges.js`)** The Steganography arena follows an insider whistleblower leaking files out of *EvilCorp*. Every question has a downloadable evidence file hosted in `/public/steg/`.

- **Q1: Evil Terms of Service (Warm-Up 1):** Players download `Evil_Terms_Of_Service.docx`, which says you give up all rights by scrolling to the bottom. Selecting the whole page (`Ctrl+A`) and changing the font color to a dark shade reveals hidden white-on-white text at the end: **CAT{but_i_refuse}**. *How it teaches:* It shows how files can hide invisible text streams, which connects directly to how people hide invisible prompts inside PDFs and webpages today.

- **Q2: Evil Mug (Warm-Up 2):** Players download `evil_mug.jpg`, a harmless-looking photo of a company mug. Checking the file's EXIF metadata (via Windows `Properties -> Details` or online at `exif.tools`) reveals a note in the `Comments` field: **CAT{detailed_search}**. *How it teaches:* It proves that image files carry invisible text headers alongside the visible picture.

- *— Dotted Line Divider (Start of 5-Stage Whistleblower Chain) —*

- **Q3: BLACKBOOK (Chain Stage 1):** The file `blackbook.png` looks like a completely blank black rectangle. Opening it in **Photopea** (`photopea.com`) and dragging the Brightness/Levels slider all the way up illuminates dark text (`RGB(4,4,4)` on `RGB(0,0,0)`) that reads **CAT{l1ghtw0rks}**. *How it teaches:* What looks pitch black to human eyes still holds different pixel numbers in computer memory, which is why poorly redacted screenshots often leak secrets when brightened.

- **Q4: Gimmick Boss (Chain Stage 2):** In `gimmick_boss.jpg`, checking the EXIF comment first gives the clue `password:lifeisajoke`. Uploading the photo to an online

**Steghide** extractor with the password `lifeisajoke` pulls out a hidden text file ending with a line of 8-bit binary. Converting that binary to ASCII gives **CAT**{**whysoserious**}. *How it teaches:* Players learn how Steghide hides encrypted files inside the Least Significant Bits (LSB) of an image and how binary bytes map to letters.

- **Q5: EvilChair (Chain Stage 3):** The photo `evilchair.jpg` shows an office chair, but opening the file in the online hex editor **HexEd.it** (or running `strings`) and scrolling to the very bottom shows extra text pasted right after the JPEG End-Of-Image bytes (`FF D9`): **CAT**{**saveaseat**}. *How it teaches:* Normal photo viewers stop reading a file the instant they hit the `FF D9` closing bytes, meaning whole messages or zip files can be piggybacked onto the tail end of an image.

- **Q6: EvilCorp Memory Wipe (Chain Stage 4):** Players download the audio broadcast `memory_wipe.wav`. The prompt warns them not to listen to it, but to *look* at it. Uploading the audio to an online **Spectrogram Viewer** (like Academo) displays the sound frequencies between 5 kHz and 15 kHz, where the sound waves visually spell out **cat**{**waverunner**}. *How it teaches:* Seeing words drawn inside sound waves is an unforgettable moment that introduces students to audio forensics and Fast Fourier Transforms (FFT).

- **Q7: tryfindme (Chain Stage 5):** The final image, `tryfindme.jpg`, has a metadata note saying the password is hidden inside the picture itself. Brightening the image in Photopea reveals a faint `0004` below the silhouettes. Using `0004` as the Steghide password extracts a hidden file containing `Q0FUe3lvdWZvdW5kbWUhfQ==`, which decodes from Base64 into **CAT**{**youfoundme!**}. *How it teaches:* It ties together metadata inspection, brightness shifting, Steghide extraction, and Base64 decoding into one smooth workflow.

**C. Cryptography Arena (`CryptoChallenges.js`)** The Cryptography arena (*Operation Blackbox*) targets how corporations misuse "encryption" as a marketing buzzword while relying on weak keys and unsalted hashes behind the scenes.

- **Q1: The Leaked Executive Memo (Warm-Up 1):** EvilCorp directors rotated a memo by 13 letters: `PNG{rivypbec_cevinpl_vf_n_zlgu}`. Running **ROT13** in CyberChef shifts each letter by 13 places to reveal **CAT**{**evilcorp_privacy_is_a_myth**}. *How it teaches:* Introduces alphabet rotation and shows why simple substitution offers zero real security.

- **Q2: The Fake Telemetry "Encryption" (Warm-Up 2):** EvilCorp claims their app telemetry is encrypted, but the string ends with ==: `Q0FUe3RlbGVtZXRyeV9pc19ub3RfZW5jcnlwdGVkfQ==`. Decoding it with **From Base64** in CyberChef reveals **CAT**{**telemetry_is_not_encrypted**}. *How it teaches:* Clears up the huge confusion between encoding (which anyone can reverse with no key) and true encryption. After solving this, students start recognizing Base64 strings ending in = inside real web URLs and cookies.

- *— Dotted Line Divider (Start of 5-Stage Cryptographic Chain) —*

- **Q3: The Whistleblower's Hex Packet (Chain Stage 1):** Players decode a raw hexadecimal dump (`43 41 54 7b ...`) using CyberChef's **From Hex** tool to get **CAT**{**whistleblower_key_is_SHADOWCORP**}. This gives them the secret passphrase `SHADOWCORP`.

- **Q4: The Encrypted Boardroom Dispatch (Chain Stage 2):** Players take the key `SHADOWCORP` from Q3 and use it in CyberChef's **Vigenère Decode** tool on the ciphertext `UHA{klsgor_ewbrh_gg_ggnso}` to unlock **CAT**{**caesar_shift_is_seven**}. This reveals the shift number `7` for the next stage.

- **Q5: Unlocking the Shadow Vault (Chain Stage 3):** Using the shift of `7` from Q4, players shift `JHA{ahynoa_chbsa_jlyilybz}` backward by 7 positions to reveal **CAT**{**target_vault_cerberus**}, uncovering the vault codename `cerberus`.

- **Q6: Cracking the CISO's Password Hash (Chain Stage 4):** The vault is locked by an unsalted MD5 hash: `f6be3f2408481885304a362deafa168a`. Pasting this hash into **CrackStation** (`crackstation.net`) searches a massive precomputed rainbow table and cracks the password in less than a second: `cerberus123` (**CAT**{**cerberus123**}). *How it teaches:* Watching a 32-character cryptographic hash break instantly because the executive added `123` to a word without a cryptographic salt permanently changes how students choose their own passwords.

- **Q7: Neutralizing the Surveillance Master Payload (Chain Stage 5):** The final payload is encrypted with repeating-key XOR using the password from Q6 (`cerberus123`) and wrapped in Base64. Building a 2-step recipe in CyberChef (`From Base64` followed by `XOR` with key `cerberus123`) unlocks the master flag: **CAT**{**evilcorp_surveillance_neutralized_2026**}.

**D. Web Security Curriculum (`WireframeChallenges.js`)** The fourth domain covers 7 foundational web vulnerabilities in the same 2+5 format: inspecting HTML source comments (Q1), HTTP request verbs (Q2), re-enabling disabled HTML buttons via Inspect Element (Q3), tampering with session cookies from `role=user` to `role=admin` (Q4), changing user ID numbers in URLs to exploit IDOR (Q5), directory path traversal (Q6), and bypassing login forms with SQL Injection `' OR 1=1 --` (Q7).

## 3. Conclusion

Building **Cyber Awareness Academy** was a massive learning curve for our entire team, both in terms of software engineering and understanding how regular people think about technology. What started as a simple college project idea turned into a full platform that genuinely changes how newcomers look at their digital privacy.

### 3.1 Key Learnings

On the technical side, we learned how to wire a full-stack React 19 application to a Node.js REST backend and manage an embedded SQLite3 database (`ctf.db`) so that it seeds itself automatically without manual setup. We also learned a lot about state management when we had to fix real-time score synchronization so that solving a question inside an arena immediately updates the progress bar on the main dashboard cards. On the design side, stripping away our early rounded boxes and fake macOS window dots in favor of a clean Gruvbox Dark terminal theme taught us that simple, intentional design always beats flashy clutter. Most importantly, watching non-technical friends test our 2+5 chains proved our core idea: when you remove the annoying setup barriers and give people a good story with a clear textbook right next to the puzzle, anyone can learn cybersecurity.

### 3.2 Future Scope and Improvements

While the current version works great as an offline local implementation, there are several things we plan to improve next:

1. **Hosting on Our Own Server:** Right now, the user runs `node server.js` and `npm start` locally. Our immediate next step is to package the app in Docker and host it on our personal Linux server behind Nginx so anyone can play through a public web link.

2. **Live Sandboxed Web Security Targets:** We want to upgrade the 7 Web Security challenges from interactive concept cards into live, isolated backend routes (such as a real sandboxed SQLite login box for SQL injection and a live cookie-tampering endpoint).

3. **Built-In Browser Decoders:** Adding a small built-in Base64/Hex/ROT13 converter and EXIF viewer directly inside our React UI would save players from even having to open a second browser tab.

4. **Dynamic Per-User Flags:** Generating unique flags for each student session will allow professors to use the platform for graded college lab exams without worrying about students copying flags from each other.

### 3.3 Open-Source Sources and Inspirations

Our project was heavily inspired by open-source tools and communities that make security knowledge free for everyone. We drew direct inspiration from **GCHQ's CyberChef** for our browser-based decoding pipelines, **Carnegie Mellon's picoCTF** and **OverTheWire** for our progressive puzzle chains, and **Pavel Pertsev's Gruvbox** color palette for our terminal aesthetic. We also rely on free web utilities including Photopea, HexEd.it, CrackStation, Exif.tools, and the Academo Spectrum Analyzer, alongside the open-source React and SQLite ecosystems.

## References


J. Vykopal, V. Švábenský, and E. Chang, "Benefits and pitfalls of using capture the flag games in university courses," in *Proc. 51st ACM Technical Symp. Computer Science Education (SIGCSE)*, Portland, OR, USA, 2020, pp. 752–758.

P. Chapman, J. Burket, and D. Brumley, "picoCTF: A game-based computer security competition for high school students," in *Proc. USENIX Summit Gaming, Games, and Gamification in Security Education (3GSE)*, San Diego, CA, USA, 2014, pp. 1–10.

OverTheWire Security Community, "OverTheWire: Bandit and Natas Wargames," 2024. [Online]. Available: https://overthewire.org/wargames/

TryHackMe, "Hands-on cyber security training through browser-based labs," 2024. [Online]. Available: https://tryhackme.com/

Hack The Box, "Cybersecurity training and penetration testing labs," 2024. [Online]. Available: https://www.hackthebox.com/

V. Švábenský, J. Vykopal, M. Cermak, and M. Laštovička, "Enhancing cybersecurity skills by creating serious games," in *Proc. 23rd Annu. ACM Conf. Innovation and Technology in Computer Science Education (ITiCSE)*, Larnaca, Cyprus, 2018, pp. 194–199.

S. K. Chng, H. Lu, A. Kumar, and D. Yau, "Hacker types, motivations and strategies: A comprehensive framework," *Computers in Human Behavior Reports*, vol. 5, p. 100167, Mar. 2022.

Government Communications Headquarters (GCHQ), "CyberChef: The Cyber Swiss Army Knife," GitHub Repository, 2024. [Online]. Available: https://github.com/gchq/CyberChef

P. Pertsev, "Gruvbox: Retro groove color scheme for Vim and terminal emulators," GitHub Repository, 2012. [Online]. Available: https://github.com/morhetz/gruvbox

A. Hern, "Fitness tracking app Strava gives away location of secret US army bases," *The Guardian*, Jan. 28, 2018. [Online]. Available: https://www.theguardian.com/world/2018/jan/28/fitness-tracking-app-gives-away-location-of-secret-us-army-bases

Open Web Application Security Project (OWASP), "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

S. Hetzl and P. Mutzel, "A graph-theoretic approach to steganography," in *Communications and Multimedia Security (CMS)*, Lecture Notes in Computer Science, vol. 3677. Berlin, Germany: Springer, 2005, pp. 119–128.